

12.476

RAVULA SHASHANK REDDY - EE25BTECH11047

October 12, 2025

Question

If

$$\mathbf{AB} = \mathbf{I},$$

then:

 $\mathbf{B} = \mathbf{A}^T$

 $\mathbf{A} = \mathbf{B}^T$

 $\mathbf{B} = \mathbf{A}^{-1}$

 $\mathbf{B} = \mathbf{A}$

Solution

Given:

$$\mathbf{AB} = \mathbf{I} \quad (1)$$

Multiply both sides by $\mathbf{A}^{-1}$:

$$\mathbf{A}^{-1}(\mathbf{AB}) = \mathbf{A}^{-1}\mathbf{I} \quad (2)$$

$$\Rightarrow (\mathbf{A}^{-1}\mathbf{A})\mathbf{B} = \mathbf{A}^{-1} \quad (3)$$

$$\Rightarrow \mathbf{I}\mathbf{B} = \mathbf{A}^{-1} \quad (4)$$

$$\Rightarrow \mathbf{B} = \mathbf{A}^{-1} \quad (5)$$

$$\boxed{\mathbf{B} = \mathbf{A}^{-1}}. \quad (6)$$

C Code

```
#include <stdio.h>

#define N 3 // size of the square matrices

// Function to multiply two matrices
void multiply(int A[N][N], int B[N][N], int C[N][N]) {
    for(int i=0; i<N; i++) {
        for(int j=0; j<N; j++) {
            C[i][j] = 0;
            for(int k=0; k<N; k++)
                C[i][j] += A[i][k] * B[k][j];
        }
    }
}

// Function to print a matrix
void printMatrix(int M[N][N]) {
    for(int i=0; i<N; i++) {
```

```
        for(int j=0; j<N; j++)
            printf("%d", M[i][j]);
        printf("\n");
    }
}

int main() {
    // Example matrices
    int A[N][N] = {{1,0,0},{0,1,0},{0,0,1}};
    int B[N][N] = {{1,0,0},{0,1,0},{0,0,1}}; // B = inverse of A
        (identity in this case)
    int C[N][N];

    multiply(A, B, C);

    printf("Matrix AB:\n");
    printMatrix(C);
}
```

```
// Check if C is identity
int isIdentity = 1;
for(int i=0;i<N;i++)
    for(int j=0;j<N;j++)
        if((i==j && C[i][j]!=1) || (i!=j && C[i][j]!=0))
            isIdentity = 0;

if(isIdentity)
    printf(Hence, B = A^-1\n);
else
    printf(B is NOT the inverse of A\n);

return 0;
}
```

```
import numpy as np

# Example 3x3 matrix A
A = np.array([[2, 1, 1],
              [1, 2, 1],
              [1, 1, 2]])

# Suppose B satisfies  $AB = I$ 
# Compute  $B = \text{inverse of } A$ 
B = np.linalg.inv(A)

# Verify  $AB = I$ 
AB = np.dot(A, B)
```

```
print(Matrix A:\n, A)
print(\nMatrix B (A^-1):\n, B)
print(\nAB:\n, AB)

# Check if AB is approximately identity
if np.allclose(AB, np.eye(3)):
    print(\nHence, B = A^-1)
else:
    print(\nB is NOT the inverse of A)
```



```
import ctypes
import numpy as np

N = 3

# Load the shared library
lib = ctypes.CDLL('./libmatrix.so')

# Define argument types for verify_inverse
lib.verify_inverse.argtypes = [ctypes.POINTER(ctypes.c_double)
                                for _ in range(2)]
lib.verify_inverse.restype = ctypes.c_int

# Example matrix A
A = np.array([[2.0, 1.0, 1.0],
              [1.0, 2.0, 1.0],
              [1.0, 1.0, 2.0]])
```

```
# Compute B = inverse of A using numpy
B = np.linalg.inv(A)

# Convert numpy arrays to ctypes
A_ctypes = A.ctypes.data_as(ctypes.POINTER(ctypes.c_double))
B_ctypes = B.ctypes.data_as(ctypes.POINTER(ctypes.c_double))

# Call C function
result = lib.verify_inverse(A_ctypes, B_ctypes)

if result:
    print(AB = I, hence B = A-1 (Verified in C via ctypes))
else:
    print(B is NOT the inverse of A)
```